



МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ТЕХНОЛОГИЧЕСКОГО ОБРАЗОВАНИЯ
Кафедра информационных технологий и электронного обучения

ОТЧЁТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ ПРАКТИКИ
технологическая (проектно-технологическая)

по направлению «09.03.01 – Информатика и вычислительная техника»
(профиль: «Технологии разработки программного обеспечения и обработки
больших данных»)

Зав. кафедрой ИТиЭО д.п.н., проф.

(Власова Е. З.)

Руководитель: доцент каф. ИТиЭО

(Жуков Н. Н.)

Студент 2 курса

(Борисов Артур Александрович)

Санкт-Петербург

2026

Введение

Учебная технологическая практика проходила с 1 по 28 сентября 2026 года в НО «Ассоциация любителей гребного спорта». Целью работы была разработка учебного консольного TODO-приложения на языке Go, которое моделирует учет организационных задач: создание записи, просмотр списка, удаление задачи и изменение ее состояния на «выполнено».

Проект выбран с учетом задач подготовки к стажировке Go-разработчика. В ходе практики внимание уделялось не только написанию кода, но и формулированию требований, настройке рабочего места, разделению пользовательского ввода и основной логики, автоматическим тестам, работе с Git и подготовке документации.

1 Анализ предметной области и требований

В деятельности спортивной организации регулярно возникают поручения, связанные с подготовкой мероприятий, взаимодействием с участниками, документами и материальным обеспечением. Для учебного проекта эта деятельность была интерпретирована как набор задач, каждой из которых можно назначить идентификатор, название, описание и состояние выполнения.

Приложение рассматривается как учебный прототип, а не как внедренная информационная система организации. Оно должно запускаться из командной строки, поддерживать понятный набор операций и позволять проверять основную логику независимо от ввода пользователя.

- добавление новой задачи с уникальным идентификатором, названием и описанием;
- просмотр текущего набора задач и их состояния;
- удаление задачи по идентификатору;
- перевод существующей задачи в состояние «выполнено»;
- предсказуемое поведение при обращении к отсутствующему идентификатору;
- проверка логики стандартными средствами тестирования Go.

2 Рабочее место и программные средства

Для разработки использовались язык Go, редактор Visual Studio Code, командная строка и система контроля версий Git. Такой набор подходит для учебного проекта: Go предоставляет компилятор, форматирование и стандартный пакет testing; Visual Studio Code объединяет редактирование, навигацию и запуск команд; Git фиксирует последовательные этапы разработки.

2.1 Настройка среды

Рабочий процесс включает проверку установленной версии Go, создание модуля проекта, форматирование исходных файлов, сборку, запуск приложения и выполнение

тестов. Репозиторий используется для просмотра изменений и сохранения логически завершенных этапов отдельными коммитами.

3 Проектирование приложения

Основной сущностью проекта является задача. Она содержит идентификатор, текстовые сведения и признак выполнения. Для доступа к задаче по известному идентификатору применяется `map`, где ключом выступает идентификатор, а значением — структура задачи. Это позволяет непосредственно обращаться к нужной записи без последовательного перебора всего набора.

Код приложения находится в файле `first.go`, а автоматические тесты — в `first_test.go`. Начальные данные теста, вызов проверяемой функции, ожидаемый результат и фактический результат рассматриваются отдельно. Такое разделение делает причину `PASS` или `FAIL` понятной.

3.1 Основные операции

- создание и добавление задачи в коллекцию;
- вывод перечня задач;
- поиск и удаление записи по ключу `map`;
- получение копии структуры, изменение состояния и запись обновленного значения обратно в `map`;
- передача пользовательского ввода в отдельную функцию основной логики.

4 Тестирование

Для проверки операций подготовлены модульные тесты на стандартном пакете `testing`. Каждый тест создает контролируемое начальное состояние, вызывает одну проверяемую операцию и сравнивает фактическое состояние с ожидаемым. Отдельное внимание уделяется существующим и отсутствующим идентификаторам.

Намеренный `FAIL` используется как учебный прием: изменение ожидаемого результата или временное исключение записи обновленной структуры обратно в `map` показывает, какую ошибку способен обнаружить тест. После объяснения причины расхождения код возвращается к корректному состоянию, а набор тестов запускается повторно.

4.1 Итоговая фиксация результатов

На этапе завершения `TODO v1.0` работоспособность основной логики и модульных тестов была проверена автором проекта. При подготовке электронного портфолио исходный код и тесты перенесены без повторного запуска и без изменений. Репозиторий проекта опубликован по адресу: <https://github.com/arturbro6/HerzenPractice-09.2026>.

5 Работа с Git и документацией

Изменения проекта фиксируются небольшими содержательными коммитами. В репозитории должны быть размещены исходный код, файл тестов, краткое руководство пользователя и материалы отчета. Руководство должно объяснять назначение приложения, требования к запуску, команды меню и примеры основных операций.

6 Анализ вариантов дальнейшего развития

Базовая консольная версия решает учебную задачу, но не исчерпывает возможное развитие проекта. В рамках практики подготовлен анализ следующих вариантов; их перечисление не означает, что все они реализованы к моменту сдачи.

- сохранение задач в JSON-файл, чтобы данные не исчезали после завершения программы;
- введение явной обработки ошибок для неверного ввода и отсутствующих записей;
- расширение набора тестов граничными случаями и табличными сценариями;
- создание HTTP API и перенос операций из консольного меню в обработчики запросов;
- подключение базы данных после появления требований к долговременному хранению и параллельной работе;
- добавление алгоритмических упражнений и измерений только там, где возникает практическая потребность.

Рациональная последовательность развития: сначала стабилизировать базовую логику и тесты, затем добавить файловое хранение и обработку ошибок, после этого изучить HTTP и только затем подключать базу данных. Такой порядок уменьшает количество новых конструкций на одном этапе и позволяет подтверждать каждое изменение тестами.

7 Электронное портфолио

Материалы практики опубликованы в открытом GitHub-репозитории. Для каждого пункта индивидуального задания подготовлены версии DOCX и PDF, а ссылки и QR-коды ведут непосредственно на соответствующие PDF-файлы.

Репозиторий: <https://github.com/arturbro6/HerzenPractice-09.2026>

Пункт и материал	Ссылка	QR-код
1.1 Анализ предметной области Форма отчётности: Краткое описание предметной области и перечень требований к приложению	Открыть материал	

Пункт и материал	Ссылка	QR-код
1.2 Обзор программных средств Форма отчётности: Конспект обзора программных средств	Открыть материал	
1.3 Настройка среды Форма отчётности: Описание настройки среды и подтверждение запуска проекта	Открыть материал	
2.1 Проектирование приложения Форма отчётности: Описание структуры приложения и исходный код	Открыть материал	
2.2 Добавление и просмотр Форма отчётности: Исходный код и примеры работы программы	Открыть материал	
2.3 Удаление и завершение Форма отчётности: Исходный код и описание реализованных функций	Открыть материал	
2.4 Модульное тестирование Форма отчётности: Файл автоматических тестов и описание проверяемых сценариев	Открыть материал	
2.5 Работа с Git Форма отчётности: История коммитов и краткое описание порядка работы с Git	Открыть материал	

Пункт и материал	Ссылка	QR-код
2.6 Руководство пользователя Форма отчётности: Руководство пользователя	Открыть материал	
2.7 Варианты развития Форма отчётности: Аналитическая записка с вариантами дальнейшего развития	Открыть материал	

Заключение

В ходе практики подготовлен учебный проект, связывающий реальные потребности учета организационных поручений с освоением разработки на Go. Работа охватывает анализ требований, настройку среды, проектирование структуры данных, реализацию основных операций, автоматическое тестирование, контроль версий и пользовательскую документацию.

К завершению практики 28 сентября подготовлены отчёт, индивидуальное задание, характеристика и электронное портфолио. В портфолио размещены исходный код, тесты, руководство пользователя и отдельные материалы по каждому пункту индивидуального задания.

Руководитель практики _____

Задание выполнил _____ Борисов А. А.